Name: Logan M Spinali
WSU ID: 011892661

CPT_S 360 - PA 3 Report

Since my implementation of the kmlab module takes place entirely in one module, I will use this report to discuss the implementation of said module. There are a few primary sections within it. The first section that is important to talk about is the initialization and exit process of the module. Initialization begins by making the /proc directory and file for communication with userspace activity. These instantiations use the proc_mkdir and proc_create functions respectively and also include improper open checks. Next, the program begins the timer, using the timer_setup function to connect it to the callback function “wakeup” and then starting the timer with a value of 5000 milliseconds to jiffies (five seconds).

The exit process is more interesting since it works with the mutex lock I set up to ensure nothing breaks as it cleans up memory. The exit process begins by stopping the timer and work queued for updating the cpu times in the list and removing the proc files, then it enables the lock. Once the lock is enabled, the exit process safely deletes each item in the process list and disables the lock before exiting safely. The lock is extremely important, as without it, if a process tried to access the list as it was being deleted, the kernel could crash, additionally it is important to disable the timer and work queue activity before exiting so nothing crashes from an invalid callback function access.

Before discussing the next section, I want to address why I chose to use a mutex lock over a spinlock. The primary reason I chose to use a mutex lock was the fact that they have no cpu usage during sleep times. Because I needed to lock down all list access operations, with a mutable list size and an unknown maximum length of the list, I determined that it was important to not have high cpu usage from the waiting threads while the list traversal was taking place. In the case of the simple testing cases, there have only been a couple of processes active at once, meaning the context switching cost is almost certainly more expensive than the waiting cost of spinlocks. However, users of this module are likely to connect many processes to it, and therefore raise the list traversal times that lock down the program.

The next important aspect of my implementation are the procfile read and write callback functions. Starting with procfile_write, the implementation had a few unique quirks as compared to reference code in the provided links. Firstly, I had to learn to use the kernel library version of stoi (kstrtouint in this case) and the function copy_from_user. These functions were quite manageable, despite their confusing headers in documentation, and most of my trouble came from the second part of the implementation: determining where to activate and deactivate the mutex lock. Initially I had a mutex lock activation in both the write function and the add_process function, which caused a deadlock situation. I decided in the end to put my lock activations only in the base functions that were calling others (write, read, update_cpu_time, etc). Additionally the lock was used in the write functions to protect the proc file state while reading or writing.

The read function for the procfile operations utilizes the seq_file (sequential file) API to make file access easier from multiple locations since the seq_read function allows copying to userspace. (this ends up avoiding the copy_to_userspace function for a more streamlined solution). When the status procfile is opened, it sets up the seq_file struct to open the procfile for reading, then when the read action is called, the procfile_show function prints the pid and cputime values in the list, formatting them for readability.

The next section worth talking about is the linked list usage. Linux uses a circular doubly linked list for its implementation, and it includes many helpful functions for utilizing the list (especially the for_each loops). The program contains a static list head that points to the first proc_node. Each proc_node contains a list_head (linux’s node indicator) that can be used to reference the process node it is contained

within. For list traversal without removal, the list_for_each_entry loop can be used to great effect, allowing developers to go through the whole list easily. During the node removal loop (when updating cpu times), we have to use list_for_each_entry_safe. This is very similar to the other loop, but stores the next position in the loop so node removal can be performed without worries.

The timer setup is very simple, beginning with a five second countdown before calling the wakeup function. In wakeup, it uses the schedule_work function to add the work of updating the cpu times to the queue, then re-arms the timer for five seconds using the mod_timer function. When the work is run, the function checks if the list is not empty, then loops through each process and gets the cpu use time for the process. Additionally, on modification of these values I print out the current state of the list to the dmesg output for debugging and monitoring purposes. When a pid is not present or invalid in the get_cpu_use function, the process with that id is considered terminated and is removed from the list.

Finally, the userapp is a very simple program that utilizes processor power by incrementing a counter every loop for a defined number of seconds. The program begins by registering its pid with the module (opening the proc file, fprintf to write its pid), then runs a loop for the input number of seconds (default of 10), and lastly printing the list of processes in the module right before it terminates (as well as the number it reached on the counter).

```
vortexcoregaming@ubuntuvm:~/Documents/CPT_S360/PA_3_Parent_Folder/PA_3/helper_files$ sudo insmod kmlab.ko
vortexcoregaming@ubuntuvm:~/Documents/CPT_S360/PA_3_Parent_Folder/PA_3/helper_files$ ./userapp 15

PID 75069 reached value: 2316311249
==Processes==
PID: 75069 -> CPU Time: 10080000000

vortexcoregaming@ubuntuvm:~/Documents/CPT_S360/PA_3_Parent_Folder/PA_3/helper_files$
```

```
vortexcoregaming@ubuntuvm:~/Documents/CPT_S360/PA_3_Parent_Folder/PA_3/helper_files$ ./userapp & ./userapp 16
[1] 75319

PID 75319 reached value: 2064261410
==Processes==
PID: 75320 -> CPU Time: 316000000

PID: 75319 -> CPU Time: 292000000


PID 75320 reached value: 3962019314
==Processes==
PID: 75320 -> CPU Time: 3252000000

[1]+  Done                    ./userapp
vortexcoregaming@ubuntuvm:~/Documents/CPT_S360/PA_3_Parent_Folder/PA_3/helper_files$
```

AI Disclosure: I utilized Claude for double checking if there were any memory leaks or deadlock situations remaining in the project. It responded by correctly identifying a case where the program could reach a deadlock during the exit process if a proc_read/write was waiting for the lock and the exit had already locked and deleted the proc files. I made the corresponding fix and proceeded with the project. Additionally, the automatic popup Gemini AI was used as a syntax and concept aid for the linux-kernel-specific functions and libraries.